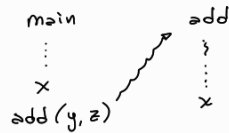
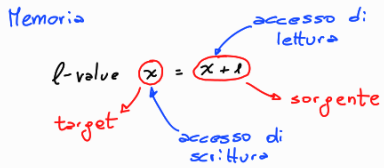


Espressioni $\rightarrow$ con id costanti } $\rightarrow$ Ambiente
 Dichiarazioni $\rightarrow$ con id costanti } (associazione id - oggetto denotato)

Concetto di: tipo (tipo dell'oggetto denotato)

$\hookrightarrow$ Semantica statica che verifica i vincoli contestuali (vincoli di tipo)



$a[i] := x + 1$

$\hookrightarrow$ LOCAZIONE (astrazione della cella di memoria)

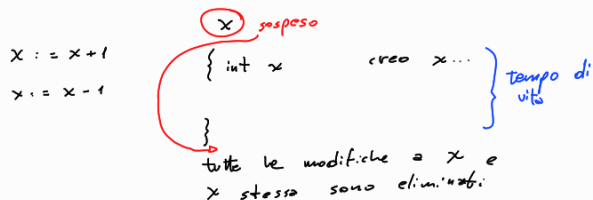


Memoria = insieme di associazioni tra locazioni e valori memorizzabili

valori $\rightarrow$ esprimibile: (se rappresentabile mediante espressioni)
 $\rightarrow$ denotabili: (se riferibili mediante identificatori)
 $\rightarrow$ memorizzabili: (se possono essere associati a locazioni)

La memoria serve per catturare le trasformazioni irreversibili generate dal programma

L'ambiente cattura trasformazioni reversibili:



MEMORIA

Mem insieme di memorie (Associazioni tra locazione e valore)

Loc insieme di locazioni

MVal insieme di valori

$$\sigma \in Mem \quad \sigma: Loc \rightarrow MVal$$

$$L \in Loc$$

$$\forall \sigma \in L. \sigma: l \mapsto v \in MVal$$

$$Mem_L = L \rightarrow MVal = DVal \cup Loc$$

$$L \subseteq_f Loc$$

$$Mem = \bigcup_{L \subseteq_f Loc} Mem_L$$

insieme di associazioni da un qualunque insieme finito di Loc ad un insieme di valori (memorizzati)

$$DVal = EVal \cup Loc \rightsquigarrow N \cup B \cup Loc$$

$$MVal = N \cup B$$

Aggiornamento di una memoria

$$\sigma = [\sigma']$$

$$\sigma, \sigma' \in \text{Mem} \quad \sigma: L \quad \sigma': L'$$

(L e' un dominio di σ) (L' dominio di σ')

$$\sigma'' = \sigma[\sigma'] \in \text{Mem}$$

$$\forall \ell \in L \cup L' . \sigma''(\ell) = \begin{cases} \sigma'(\ell) & \ell \in L' \\ \sigma(\ell) & \ell \in L \setminus L' \end{cases}$$

Aggiungiamo alle dichiarazioni: la dichiarazione di identificatore variabile

$$\text{Dec} : D \rightarrow \dots \mid \text{var } x : \tau = e \quad \text{Tipi} = \{\text{int}, \text{bool}, \text{intloc}, \text{boolloc}\}$$

dobbiamo aggiungere ai tipi
il tipo di locazione

$\tau_{\text{loc}} \rightarrow$ locazione di tipo τ

($\text{intloc} : \text{loc per valori interi}$
 $\text{boolloc} : \text{loc per valori booleani}$)

Semantica statica per Dec

Associa un ambiente statico

$$\text{DI}(\text{var } x : \tau = e) = \{x\} \quad \text{FI}(\text{var } x : \tau = e) = \text{FI}(e)$$

Aggiungiamo queste def. a quelle già viste

Reg. sem. statica da aggiungere: (alle dichiarazioni)

$$\frac{\Delta \vdash e : \tau}{\Delta \vdash \text{var } x : \tau = e : [x \mapsto \tau_{\text{loc}}]}$$

Semantica statica delle espressioni identificatore quando e' variabile

$$\Delta \vdash I : \tau \text{ se } \Delta(I) \in \{x, \tau_{\text{loc}}\}$$

Semantica dinamica

$\hookrightarrow$ cambia per l'aggiunta della memoria

espressioni

$$\left\{ \begin{array}{l} T = \Sigma \times \text{Mem} \rightarrow V \cup \text{Mem} \\ \text{NVB} \\ \rho \vdash e, \sigma \mapsto \langle e', \sigma' \rangle \end{array} \right.$$

la semantica dinamica delle espressioni
si porta dietro la memoria in cui
valutare l'espressione

dichiarazioni

$$\left\{ \begin{array}{l} T = D \times \text{Mem} \rightarrow \overset{T}{\text{Env}} \times \text{Mem} \\ \rho = \langle d, \sigma \rangle \mapsto \langle d', \sigma' \rangle \\ \sigma' \text{ può essere diversa da } \sigma \end{array} \right.$$

-Regola da aggiungere alle espressioni:

$$\rho \vdash \langle x, \sigma \rangle \mapsto \langle n, \sigma \rangle \quad \begin{array}{l} \rho(x) = n \quad \text{oppure } \rho(x) = \ell \in \text{Loc} \\ \text{(come prima)} \quad \sigma(\ell) = n \end{array}$$

$\begin{array}{c} \nearrow \text{costante } n \\ x \\ \searrow \text{variabile } \ell \mapsto n \end{array} \in \text{NVB}$

-Regola da aggiungere alle dichiarazioni:

$$\frac{\rho \vdash \langle e, \sigma \rangle \mapsto^* \langle n, \sigma \rangle}{\rho \vdash \text{var } x : \tau = e, \sigma \mapsto \langle [x \mapsto \ell], \sigma[\ell \mapsto n] \rangle}$$

$\ell \in \text{Loc}$ nuova locazione

Unica regola delle dichiarazioni che può modificare la memoria (side effect)

Valutazione di una espressione con id variabili

Eval : $Exp \times Mem \rightarrow Val \times Mem$

$Eval(e, \sigma) = v$ sse $\vdash \langle e, \sigma \rangle \rightarrow^* \langle v, \sigma \rangle$

Equivalenza di espressioni con id variabili

$\equiv \subseteq Exp \times Exp$

$\forall e_0, e_1 \in Exp \quad e_0 \equiv e_1$ sse $\forall \sigma. Eval(e_0, \sigma) = Eval(e_1, \sigma)$ (Es. $2+x-x \equiv \frac{2x}{x}$)

Elaborazione di dichiarazioni con id variabili

Elab : $Dec \times Mem \rightarrow Env \times Mem$

$Elab(d, \sigma) = (\rho, \sigma')$ sse $\vdash \langle d, \sigma \rangle \rightarrow^* \langle \rho, \sigma' \rangle$

certo numero di passi

Equivalenza per dichiarazioni con id variabili

$\equiv \subseteq Dec \times Dec$

$\forall d_1, d_2 \in Dec \quad d_1 \equiv d_2$ sse $\forall \sigma \in Mem$

$Elab(d_1, \sigma) = Elab(d_2, \sigma)$

Per modificare la memoria abbiamo bisogno di costrutti:

dedicati $\rightarrow$ COMANDI

↓
strumenti che abbiamo nel PL
per devotare richieste di modifica
della memoria

I comandi vengono ESEGUITI per ottenere la trasformazione
della memoria denotata.

$\rightarrow$ Assegnamento (costrutto base di modifica della memoria)

$\rightarrow$ Composizione (sequenziale)

$\rightarrow$ Controllo del flusso di esecuzione (condizionale e iterativo)

Assegnamento $x := e$,
CC Exp

Com: $C \rightarrow skip \mid x := e \mid C; C \mid \rightarrow$ composizione sequenziale

while b do C $\rightarrow$ ciclo che ripete C finché
b è valutata a vero

if b then C else C $\rightarrow$ scelta condizionale se
b è valutata a vero
si esegue il ramo then
altrimenti il ramo else

Variable che consiste
in un elemento complesso
del PL

Elementi
che la
caratterizzano

$\rightarrow$ Nome/Identificatore

$\rightarrow$ Tipo (insieme dei valori/operatori sui valori)
che caratterizza il contenuto della variabile

$\rightarrow$ Indirizzo/Localizzazione

$\rightarrow$ Scope visibilità nell'ambiente (ambiente di riferimento)

$\rightarrow$ Tempo di vita misura in termini temporali dell'esistenza
della variabile